

Capital University

Faculty of Computing & Artificial Intelligence

Software Requirements Specification (SRS)

Project: Smart Home Energy & Resource Monitor
Written by Mega Team

1. Introduction.....	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions.....	1
1.4 Project Scope	1
1.5 References	2
2. Overall Description.....	2
2.1 System Environment	2
2.2 Functional Requirements Specification.....	3
2.2.1 Login and Registration	3
2.2.2 Sensor data	3
2.2.3 Managing Appliances.....	3
2.2.4 Comparing with neighbors	4
2.2.5 Anomalies	4
2.2.6 Goal Progress	4
2.3 User Characteristics.....	5
2.4 Non-Functional Requirements.....	5
2.5 NFR Categories Selected	5
2.6 NFR Specification with Fit Criteria	6
2.7 Impact of NFRs on System Architecture	8
2.7.1 Performance.....	9
2.7.2 Security	9
2.7.3 Reliability.....	9
2.7.4 Usability.....	9
2.7.5 Maintainability	9
2.7.6 Scalability.....	9
2.7.7 Portability	10
3. Requirement Specification	10
3.1 External Interface Requirements	10
3.2 Functional Requirements.....	11
3.2.1 Login and Registration.....	11
3.2.2 Sensor Data	11
3.2.3 Managing Appliances.....	12

3.2.4	Comparing with Neighbors	12
3.2.5	Anomalies Detection.....	13
3.2.6	Goal Progress	13

Revision History

Name	Date	Reason For Changes	Version
Tarek	5/4/2026	Initial version of SRS document	1.0
Tarek	8/4/2026	Some updates about Use Cases	1.1
Tarek	10/4/2026	Updates on use cases	1.2
Tarek	14/10/2026	Last version	2.0

1. Introduction

1.1 Purpose

This document provides a detailed description of the Smart Home Energy & Resource Monitor System. It outlines the system's functional and non-functional requirements, system features, and constraints.

The purpose of this SRS is to serve as a reference for developers, designers, and stakeholders to understand how the system should behave and be implemented.

1.2 Document Conventions

This document follows standard Software Engineering conventions:

- *Headings are numbered hierarchically (e.g., 1, 1.1, 1.2).*
- *Bold text is used for important terms.*
- *Italics are used for emphasis or definitions.*
- *"Shall" indicates for mandatory requirements.*
- *"Should" indicates for recommended features.*

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- **Developers** → to implement the system.
- **UI/UX Designers** → to design the interface.
- **Project managers** → to track progress.
- **Testers** → to validate system functionality.

Reading suggestion:

- Developers should focus on **Functional Requirements**.
- Testers should focus on **System Features & Constraints**.
- Managers should review **Scope and Overview**.

1.4 Project Scope

The Smart Home & Resource Monitor System is a web-based dashboard that simulates tracking of household energy, water, gas usage, tracking goal progress, some tips to make the cost less, comparing with neighbors,

The system provides:

- Real-time usage visualization (simulated data)
- Efficiency tips for reducing consumption
- Goal setting for minimizing bills
- Appliance health monitoring and alerts

The system aims to increase user awareness of resource consumption and promote efficient usage.

1.5 References

- IEEE 830 Software Requirements Specification Standard
- Smart home systems such as:
 - Nest Smart Home System
 - Sense Energy Monitor
- Course materials and lecture notes

2. Overall Description

2.1 System Environment

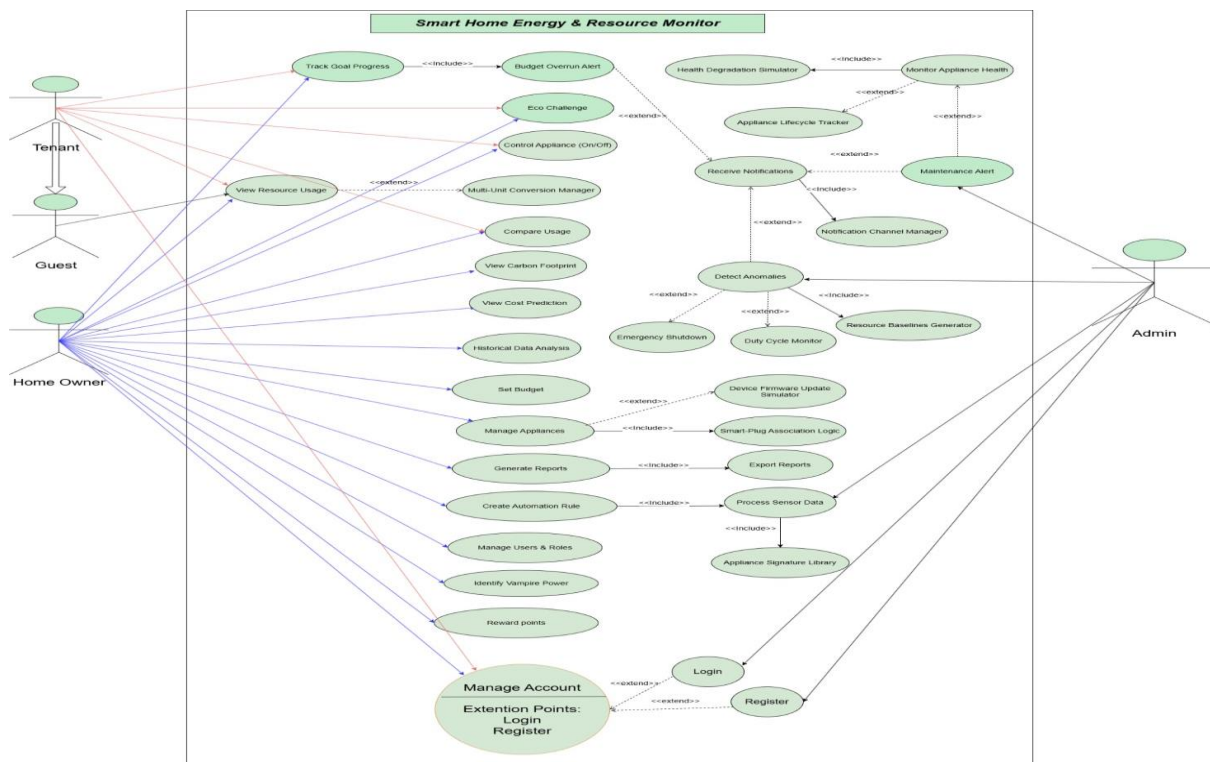


Figure 1 – System Environment

The Smart Home System has 3 actors and 1 cooperating system. Homeowner, Tenant and Guest. The Guest can only view the resource usage. Homeowner and Tenant can interact with the system through Managing account (Login, Registration), exporting reports and if there is a problem or an alert, the system will send a notification to the Homeowner or the Tenant. Tenant has fewer permissions than Homeowner, so

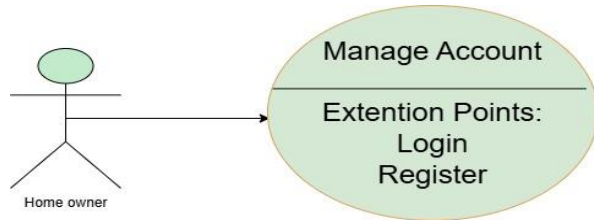
the system will send all notification to the homeowner, but not all notification will send to the tenant.

2.2 Functional Requirements Specification

This section outlines some of the use cases for the Homeowner, Tenant and System separately. The Guest has only 1 use case that is view resource usage, while homeowner and the tenant are the main actors in the system.

2.2.1 Login and Registration

Use case: Manage Account Diagram:

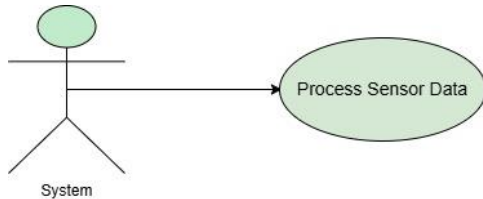


Brief description:

First the homeowner will make an account to monitor his/her apartment and see the usage of water, gas, electricity, then he/she can log in so easily to enter the account.

2.2.2 Sensor data

Use case: Process Sensor Data Diagram:

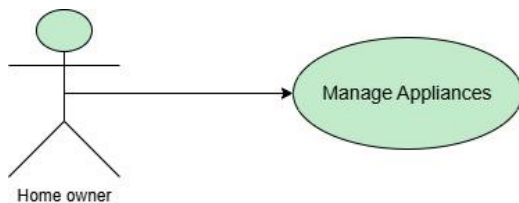


Brief description:

The system automatically generates a simulated consumption reading for each active resource type (electricity, water, gas) and persists it to the database with a simulated timestamp.

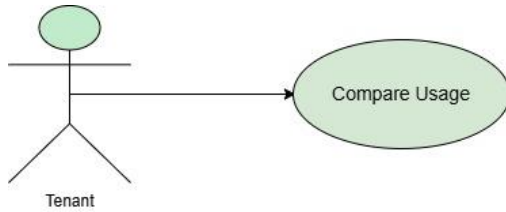
2.2.3 Managing Appliances

Use case: Manage Appliances Diagram:

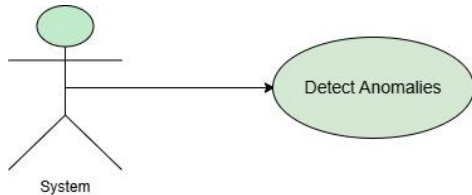


Brief description:

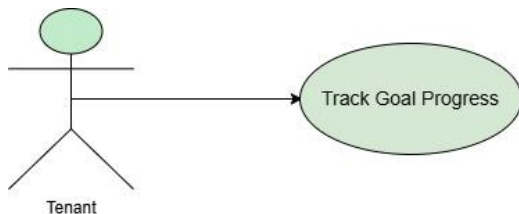
Homeowner or tenant can add/ remove or turn on/off the device remotely.

2.2.4 Comparing with neighbors**Use case: Compare Usage
Diagram:****Brief description:**

The system retrieves the household's historical consumption data and compares it against the pre-seeded Average Anonymous Household benchmark dataset. The comparison covers all three resource types (electricity, water, gas).

2.2.5 Anomalies**Use case: Detect Anomalies
Diagram:****Brief description:**

Immediately after each sensor reading is stored, the Anomaly Detection Filter automatically evaluates the new data point against three detection layers: consumption baseline deviation, ghost load detection, and short-cycling monitoring.

2.2.6 Goal Progress**Use case: Track Goal Progress
Diagram:****Brief description:**

The system continuously tracks the user's progress toward all active goals — monthly budgets, time-limited eco-challenges, and long-term sustainability milestones — and presents a unified progress view on the dashboard. After every new sensor reading, the system recalculates progress figures for all active goals and checks whether any escalation threshold or completion condition has been met. Reached milestones unlock badges, completed challenges award Eco-Credits, and budget overruns trigger the multi-stage escalation notification sequence. The Child role has a restricted view showing only their personal eco-challenge progress and Eco-Credits balance.

2.3 User Characteristics

The system has 3 actors, each one of them has permission or accesses differ from another one.

The Homeowner is the primary and most privileged user of the system. This user is assumed to have general comfort with web-based applications and smart home concepts but is not required to have any technical or engineering background. The Homeowner is responsible for the initial system configuration — including appliance inventory setup, TOU pricing tiers, budget allocation, and automation rules — and is the sole user permitted to manage other accounts, access the full audit trail, and perform data exports. It is expected that the Homeowner will interact with the system daily to review consumption trends and respond to alerts, and periodically to review monthly projections and adjust goals.

The Tenant is a secondary resident of the household who has a moderate level of engagement with the system. This user is assumed to be familiar with basic web interfaces but has no responsibility for system administration. The Tenant can monitor consumption dashboards, view their assigned room budgets, participate in eco-challenges, and export audit reports, but cannot modify global settings, create automation rules, or manage other user accounts. The Tenant is expected to interact with the system several times per week, primarily to review personal usage and challenge progress.

The Guest is a temporary or infrequent visitor to the household who requires the lowest level of system access. This user is assumed to have no prior familiarity with the system and requires no onboarding. The Guest role is strictly read-only: the user may view high-level summary dashboards but cannot interact with any configurable element, view billing information, or trigger any state change within the system. Guest sessions are expected to be short and infrequent, and the Homeowner may revoke Guest access at any time.

2.4 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the Smart Home Energy & Resource Monitor. They determine how well the system performs its functions under simulated operating conditions and are equally critical to functional requirements for platform reliability, security, and user adoption.

2.5 NFR Categories Selected

- Performance — System responsiveness under simulated user load and continuous data generation.
- Security — Protection of user credentials, sensitive simulated data, and enforced role-based access.
- Reliability — Simulation service availability, data persistence, and fault tolerance.
- Usability — Ease of use for all user roles (Home Owner, Tenant, Guest) across devices.

- Maintainability — Code quality, MVC separation, documentation, and configurability.
- Scalability — Ability to accommodate growing simulation data volume without performance degradation.
- Portability — System operability across modern browsers and screen sizes.

2.6 NFR Specification with Fit Criteria

Category	NFR Name	Specification	Fit Criterion (Testable)	Priority
Performance	Page Load	The dashboard and all primary monitoring pages must load within 3 seconds under normal simulated load.	Load test (100 concurrent users): 95% of requests complete in under 3 seconds.	MUST
Performance	Rule Evaluation Speed	The rule-based logic engine must evaluate all active automation rules within 500 ms per cycle after a new sensor reading is persisted.	Automated test: 50 active rules evaluated against a new reading — 100% complete within 500 ms.	MUST
Performance	Sensor Data Generation	The mock sensor generator must produce and persist a new data point within 1 second of its scheduled interval.	Timed test over 1,000 generation cycles: average persist latency under 1 second.	MUST
Security	Password Hashing	All user passwords must be stored using a salted cryptographic hash (bcrypt, minimum cost factor 10). No plain-text passwords may exist in the database.	Code review: zero plain-text passwords in DB. Penetration test confirms hashes only.	MUST
Security	Role Access Control	Unauthorised role endpoints must return HTTP 403 and log the attempt to the audit trail.	50 attempts to access Home Owner endpoints as Guest: 100% blocked and logged.	MUST
Security	Session Management	User sessions must expire after 30 minutes of inactivity. Session cookies must use HttpOnly and Secure flags enforced over HTTPS/TLS.	Manual test: session cookie confirms HttpOnly and Secure flags set correctly. Session expires after 30 min idle.	MUST
Security	Input	All user input fields must	Code audit: zero raw user	MUST

Category	NFR Name	Specification	Fit Criterion (Testable)	Priority
	Validation	be validated and sanitized server-side to prevent SQL injection, XSS, and CSRF attacks.	input in SQL statements. Automated scanner (e.g. SQL Map) finds no injection vulnerabilities.	
Security	Data Encryption	Sensitive fields at rest (billing info, user address, credentials) must be encrypted using AES-256 or equivalent.	Code review confirms AES-256 encryption applied to all sensitive columns. Decrypted values never stored in logs.	MUST
Reliability	Uptime	The simulation service must achieve 99% uptime during the assessed academic period.	Monitoring log shows under 7.2 hours downtime over any 30-day period.	MUST
Reliability	Data Recovery	The system must not lose more than 5 minutes of simulation data in the event of an unplanned server crash.	Crash simulation test: data gap after recovery is under 5 simulated minutes in 3 out of 3 trials.	MUST
Reliability	Simulation Recovery	The simulation service must recover automatically and resume data generation within 60 seconds of an unexpected restart.	Restart test: service resumes generating readings within 60 seconds on 3 consecutive restarts.	MUST
Usability	Onboarding	A new Home Owner must be able to complete initial setup — adding one appliance, setting a monthly budget, and creating one automation rule — within 15 minutes without assistance.	Usability test with 5 first-time users: 80% complete setup in under 15 minutes.	SHOULD
Usability	Responsive Design	All pages must render correctly from 768px (tablet) to 1920px (desktop) without layout breaks.	Manual test on Chrome/Firefox/Edge at 768px and 1440px: no layout breaks on all 12 primary views.	MUST
Usability	Alert Persistence	All alert notifications must remain visibly persistent on the dashboard until explicitly acknowledged by the user.	Manual test: alert banner remains visible after page refresh until dismissed. Verified on 5 alert types.	MUST

Category	NFR Name	Specification	Fit Criterion (Testable)	Priority
Maintainability	MVC Separation	All code must follow MVC architecture — no database queries in View files, business logic in Controllers, data access in Models only.	Code review: zero SQL calls in any View file. All DB interactions isolated to Model layer.	MUST
Maintainability	Documentation	All classes and public methods must include PHPDoc / JSDoc comment blocks describing purpose, parameters, and return values.	Documentation generator run: 100% of public methods have valid docblock comments.	SHOULD
Maintainability	Configurable Profiles	Mock sensor daily profiles must be stored as configurable data (JSON or DB records), not hard-coded values.	Code review: zero hard-coded consumption arrays in source files. Profile updates require no code change.	SHOULD
Scalability	Database Indexing	All foreign key columns and frequently queried fields (e.g. timestamp, appliance, userID) must be indexed.	EXPLAIN query analysis: zero full-table scans on queries with over 1,000 rows in the test dataset.	SHOULD
Scalability	Archive Strategy	Raw minute-level sensor data older than 30 simulated days must be archived to a separate table to maintain dashboard query performance.	Query benchmark: dashboard load time remains under 3 seconds after 90 simulated days of data accumulation.	SHOULD
Portability	Browser Compatibility	The system must function fully on the two most recent major versions of Chrome, Firefox, and Edge.	Cross-browser test matrix: all primary functions pass on Chrome, Firefox, and Edge in their latest two versions.	MUST

Priority Legend: **MUST** = Mandatory requirement for system acceptance. **SHOULD** = Strongly recommended; may be deferred if time constrained.

2.7 Impact of NFRs on System Architecture

2.7.1 Performance

Performance NFRs require database indexing on all foreign keys and timestamp fields, efficient aggregation rollups executed as background processes, and a mock-generator service operating independently of the dashboard request cycle to avoid UI blocking.

2.7.2 Security

Security NFRs mandate crypt password hashing, PDO prepared statements for all database queries, RBAC middleware enforced at the Controller layer, AES-256 encryption for sensitive fields at rest, and HTTPS/TLS for all client-server communication. The Controller layer acts as the primary security enforcement point.

2.7.3 Reliability

Reliability NFRs require the simulation service to implement write-ahead logging or equivalent checkpointing to prevent data loss on crash, automatic restart recovery within 60 seconds, and scheduled maintenance operations (archiving, aggregation) to execute as non-blocking background jobs.

2.7.4 Usability

Usability NFRs drive the interface design toward a responsive layout (768px–1920px) using a CSS framework such as Bootstrap. Alert persistence requirements mandate a global notification state managed server-side rather than relying solely on client-side session storage.

2.7.5 Maintainability

Maintainability NFRs enforce strict MVC separation: Models handle all database interactions, Controllers encapsulate business logic and rule evaluation, and Views contain only presentation logic. Mock sensor profiles stored as configurable data ensure future adjustments require no code changes.

2.7.6 Scalability

Scalability NFRs drive the schema to use indexed foreign keys, normalized tables, and an archive strategy that moves raw minute-level data older than 30 simulated days to a separate table, ensuring dashboard queries remain performant as data volume grows over long simulation periods.

2.7.7 Portability

Portability NFRs mandate compatibility with the two most recent major versions of Chrome, Firefox, and Edge. Use of a responsive CSS framework (Bootstrap or equivalent) and avoidance of browser-specific APIs ensures consistent rendering across all target environments.

3. Requirement Specification

3.1 External Interface Requirements

The system's only interaction with external systems is through simulated data sources that represent smart home devices and sensors. Since this project is a simulation, no real hardware devices are physically connected; however, the system mimics interaction with energy meters, water meters, and gas sensors. The simulated data includes energy consumption values, water usage levels, and gas usage readings, which are used by the system to generate insights and visualizations.

The Smart Home System communicates with an internal database that stores user information, historical consumption data, and system-generated recommendations. The database fields of interest include user ID, consumption records (energy, water, gas), timestamps, and user-defined goals. This data is continuously updated and retrieved to support system functionalities such as monitoring and analysis.

The system also interacts with the frontend user interface through a web-based platform. The dashboard sends user requests (such as setting goals or viewing reports) to the backend, and the backend responds with processed data in a structured format (JSON). This interaction enables dynamic updates of charts, notifications, and reports displayed to the user.

The Historical Data Aggregator function requests stored consumption data from the database and returns organized datasets for visualization and analysis. Similarly, the Resource-to-Task Translator processes input data from the system and generates actionable recommendations. All communications between system components are handled through standard HTTP/HTTPS protocols.

3.2 Functional Requirements

3.2.1 Login and Registration

FR-01	Login and Registration
Primary Actor(s)	<i>Homeowner, Tenant, Guest</i>
Stakeholders and Interest	<i>Users want secure and easy access to the system</i>

Trigger	<i>User selects login or registration</i>
Pre-conditions	<i>User has access to the system</i>
Post-conditions	<i>User account is created or user is logged in</i>
Main Success Scenario	1. User enters credentials 2. System validates data 3. User gains access to dashboard
Extensions	<i>Invalid credentials → error message</i>
Priority	<i>High</i>
Special Requirements	<i>Password encryption, input validation</i>

3.2.2 Sensor Data

FR-02	Sensor Data
Primary Actor(s)	<i>System</i>
Stakeholders and Interest	<i>Users need accurate monitoring of resource usage</i>
Trigger	<i>System receives or generates sensor data</i>
Pre-conditions	<i>System is running</i>
Post-conditions	<i>Data is stored and displayed</i>
Main Success Scenario	1. System generates simulated data 2. Data is processed 3. Data is shown on dashboard
Extensions	<i>Data error → system ignores or regenerates data</i>
Priority	<i>High</i>
Special Requirements	<i>Real-time simulation, data consistency</i>

3.2.3 Managing Appliances

FR-03	Managing Appliances
--------------	----------------------------

Primary Actor(s)	<i>Homeowner</i>
Stakeholders and Interest	<i>Homeowners want control over appliances</i>
Trigger	<i>Homeowner adds/updates/removes appliance</i>
Pre-conditions	<i>Homeowner is logged in</i>
Post-conditions	<i>Appliance list is updated</i>
Main Success Scenario	1. User selects appliance action 2. System updates appliance data 3. Changes are saved
Extensions	Invalid input → error message
Priority	<i>Medium</i>
Special Requirements	<i>Easy UI interaction</i>

3.2.4 Comparing with Neighbors

FR-04	Comparing with Neighbors
Primary Actor(s)	<i>Homeowner, Tenant</i>
Stakeholders and Interest	<i>Users want to compare efficiency with other</i>
Trigger	<i>User selects comparison feature</i>
Pre-conditions	<i>Data is available</i>
Post-conditions	<i>Comparison results displayed</i>
Main Success Scenario	1. System retrieves user data 2. System compares with benchmark data 3. Results displayed in charts
Extensions	No data → show message
Priority	<i>Medium</i>
Special Requirements	<i>Data visualization</i>

3.2.5 Anomalies Detection

FR-05	Anomalies Detection
Primary Actor(s)	<i>System</i>
Stakeholders and Interest	<i>Users want alerts for unusual usage</i>
Trigger	<i>Abnormal usage detected</i>
Pre-conditions	<i>Historical data exists</i>
Post-conditions	<i>Alert is generated</i>
Main Success Scenario	1. System analyzes usage 2. Detects anomaly 3. Sends alert to user
Extensions	<i>False detection → ignore</i>
Priority	<i>High</i>
Special Requirements	<i>Threshold-based detection</i>

3.2.6 Goal Progress

FR-06	Goal Progress
Primary Actor(s)	<i>Homeowner, Tenant</i>
Stakeholders and Interest	<i>Users want to track goals</i>
Trigger	<i>User view's goal progress</i>
Pre-conditions	<i>Goals are set</i>
Post-conditions	<i>Progress is displayed</i>
Main Success Scenario	1. System retrieves goal data 2. Calculate progress 3. Displays results
Extensions	<i>No goals → prompt users to set goals</i>
Priority	<i>Medium</i>
Special Requirements	<i>Progress visualization</i>

4. Unit Testing

The AOT Homes system includes a PHPUnit-based unit testing suite located in tests/Unit/AuthTest.php. The tests verify core authentication and calculation logic independently of the database or HTTP layer.

4.1 Testing Framework

Framework: **PHPUnit 10** | Configuration file: **phpunit.xml** | Test file: **tests/Unit/AuthTest.php**

4.2 Test Cases

Test Name	Description	Assertion	Expected Result
testTrueIsTrue	Verifies the test framework is operational	assertTrue(true)	PASS
testEmailValidation	Validates email format using regex pattern	assertMatchesRegularExpression	PASS
testPasswordValidation	Ensures password is at least 8 characters	assertGreaterThanOrEqual(8, strlen)	PASS
testLogin	Checks that login credentials match stored values	assertEquals (email & password)	PASS
testDashboardCalculation	Verifies array_sum correctly totals resource usage values	assertEquals(600, array_sum)	PASS

8.3 Test Results Summary

Total Tests: **5** | Passed: **5** | Failed: **0**

All unit tests passed successfully. The tests cover email format validation, password length enforcement, credential matching, and arithmetic aggregation used in the dashboard energy calculations.